

CONSIDERACIONES GENERALES PARA RESOLVER LOS EJERCICIOS

- No podrá ser corregido ningún programa que no compile.
- No agregar ni quitar funcionalidades.
- No modificar los formatos.
- No utilizar `getch` ya que Linux no lo acepta.
- No utilizar `strcpy`, se deberá crear una función o procedimiento propio.
- Si el examen es escrito debe ser entregado en tinta, con cada hoja correctamente numerada y en el margen superior de cada hoja se debe indicar a qué archivo corresponde dicho código.
- La nota dependerá de la eficiencia.

ÍNDICE

Etapa 1 — Arrays y matrices

1) Matriz transpuesta	2
2) Multiplicación de matrices aleatorias	2
3) Matriz de letras sin repetir + archivo	2

Etapa 2 — Punteros y memoria dinámica

4) Punteros: reescribir y comentar	3
5) Punteros con dos vectores	3
6) funcionPunteros: qué imprime	4
7) Array dinámico de aleatorios	4

Etapa 3 — Recursividad

8) Factorial recursivo	4
9) Valores entre dos enteros	4
10) Decimal a binario	4
11) Fibonacci recursivo	4
12) Sucesión por producto	5
13) Prueba de escritorio recursiva	5
14) Recursividad con punteros	5

Etapa 4 — Ordenamiento y búsqueda

15) Cuatro algoritmos de ordenamiento	6
16) Búsqueda binaria recursiva	6

Etapa 5 — TDA

17) TDA Números complejos	6
18) TDA PC y Usuario	6
19) TDA Electrodoméstico	6
20) TDA Empresa y Empleados	6
21) TDA Agencia y Vehículos	6
22) TDA Factura y Detalles	7
23) TDA Vuelo (aerolíneas)	7

Etapa 6 — Archivos

24) Personas desde archivo.txt	8
25) Unir archivos ordenados + tatetí	9

Etapa 7 — Listas, pilas y colas

26) Lista, pila y cola frente a un array	9
--	---

27) Funciones de una cola	9
28) Estructuras de cola y pila	9
29) Lista de char convertida en cola	9
30) Lista con todas las primitivas	9
31) Lista *void e inserción	9

- 1) Reescriba la matriz original A en una matriz transpuesta V:

$$\begin{array}{lcl}
 \mathbf{A} = & \begin{pmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 4 & 3 \\ 3 & 4 & 2 & 1 \end{pmatrix} & \mathbf{V} = \begin{pmatrix} 1 & 1 & 3 \\ 2 & 2 & 4 \\ 3 & 4 & 2 \\ 4 & 3 & 1 \end{pmatrix} \\
 \text{(matriz original)} & & \text{(transpuesta)}
 \end{array}$$

- 2) Crear un programa completo que permita cargar dos matrices de 6 x 6 (6 filas y 6 columnas). Las matrices tienen que ser cargadas por números aleatorios, impares, divisibles por 7 o 9 y menores a 1000. Crear también la función que permita multiplicar las dos matrices. Recuerden que el producto entre dos matrices da como resultado otra matriz de la misma dimensión. En el programa no puede faltar el “mostrar” de cada una de las 3 matrices.

A modo de ayuda dejamos este grafico sobre cómo se multiplican matrices:

$$\begin{pmatrix} 1 & -1 & 1 \\ 2 & 2 & 3 \\ -2 & -3 & -1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 4 \\ 0 & 2 & 5 \\ 1 & 3 & 0 \end{pmatrix} = \\
 = \begin{pmatrix} 1+0+1 & 0-2+3 & 4-5+0 \\ 2+0+3 & 0+4+9 & 8+10+0 \\ -2+0-1 & 0-6-3 & -8-15+0 \end{pmatrix} = \\
 = \begin{pmatrix} 2 & 1 & -1 \\ 5 & 13 & 18 \\ -3 & -9 & -23 \end{pmatrix}$$

- 3) Crear un programa que permita trabajar con un arreglo bidimensional de 6 x 6. En dicho arreglo se deberán guardar letras aleatorias sin repetir. Las letras pueden estar en minúsculas o mayúsculas, ejemplo: “A” y “a” son letras distintas. Una vez cargada la matriz se pide mostrarla. Mostrar solamente las vocales en la posición correcta. Mostrar la diagonal secundaria de la matriz y escribir la matriz en un archivo `matriz.txt`.

El `main` deberá verse sí o sí de la siguiente forma:

```

int main(){
    srand(time(0));
    int matriz[TAM][TAM];           // cargar mal la matriz anula el ejercicio
    cargarMatriz(matriz);           // con letras sin repetir
    mostrarMatriz(matriz);
    mostrarVocales(matriz);         // mostrar las vocales y # en las consonantes
    mostrarDiagonal(matriz);
    escribirMatriz(matriz);         // escribe el .txt
    return 0;
}

```

Resultado del programa	Resultado del archivo
<pre> ----MATRIZ---- i x G B t N y O p M q r I k Q U S V E P f Y h d e D a L j o A m C J s Z ----VOCALES---- i # # # # # # O # # # # I # # U # # E # # # # # e # a # # o A # # # # # ----DIAGONAL---- N q U f D A </pre>	<pre> i x G B t N y O p M q r I k Q U S V E P f Y h d e D a L j o A m C J s Z </pre>

4) Reescribir, comentar claramente y decir qué imprime el siguiente algoritmo:

```

int * puntero, i;
int x[5] = {1,2,3,4,5};
puntero = x + 4;
*(puntero-2)=9;
puntero--;
*(puntero) = 7;
puntero[1] = 11;
for ( i = 0; i<5; i++){
printf("----> %d ", *(x+i)); }

```

5) Explicar qué muestra por consola el siguiente algoritmo, realizando la prueba de escritorio pertinente y justificando su respuesta.

```

int vector1[5] = {2,4,1,1,-1};
int vector2[5] = {4,3,8,5,0};

int * puntero; // 4 4 100 8 -1

vector1[0] = vector2[0];
puntero = vector2;
vector1[3] = vector2[2];
puntero = &vector1[1];
vector2[4]= 99;
vector1[2]= 100;

for (int i = 0; i<5; i++){
    printf("--> %d\n", *puntero);
    puntero++;
}

```

- 6) Explicar claramente qué imprime el `main` siguiente que invoca a la `funcionPunteros` (o hacer una prueba de escritorio, si así lo prefiere). En caso de detectar que el programa no funciona solucionar el problema utilizando únicamente `*` y/o `&`.

```
int main(){
    float n1;
    float n2 = 9;
    funcionPunteros(&n1, &n2);
    printf("%f ----- %f\n", n1, n2);
    return 0;
}

void funcionPunteros(float * numero1, float *numero2){
    *numero1 = *numero2;
    float array[4] = {3,6,1,*numero1};
    *numero2 = *(array+2);
    if(numero2 == 1){
        *numero1 = 3;
        *numero2 = *numero1 + 6;
    }
};
```

- 7) Crear una función que permite cargar un array con números aleatorios de 2 cifras. La particularidad de la función es que debe recibir por parámetro la cantidad de elementos de un array de enteros y retorna el array, trabajando correctamente con memoria dinámica. Mostrar este array desde el `main`.

- 8) Crear una función/procedimiento recursivo/a que permita calcular el factorial de un numero entero, recordando que el factorial de 0 y de 1 es uno, al igual que de los números negativos, para los números positivos el factorial de n es:
- $$n \times (n - 1) \times (n - 2) \dots \times 2 \times 1$$

- 9) Crear un procedimiento recursivo que permita mostrar todos los valores enteros comprendidos entre un entero y otro entero.

- 10) Crear un programa recursivo que permita pasar un numero cualquiera de base decimal a base binaria.

- 11) Crear una función/procedimiento recursivo que nos permita encontrar el numero de la posición n de la sucesión de Fibonacci. Recordemos que por ejemplo el valor de la posición 6 es el valor de la posición 5 más el de la posición 4. La posición 0 y 1 están fijas con los valores 1 y 0.

Posición	0	1	2	3	4	5	6	7
Valor	0	1	1	2	3	5	8	13

- 12) Se quiere trabajar con una sucesión de números contruidos a partir de los primeros dos valores, el valor en la posición 0 es “2” y el valor en la posición 1 es “3”. Dadas estas condiciones iniciales se pueden construir todas las posiciones de la sucesión. Se sabe que cada posición viene dada por el producto de las dos posiciones anteriores.

Generar una función recursiva que retorne el valor de la posición n de nuestra sucesión.

Realizar la prueba de escritorio para $n = 6$.

Posición	0	1	2	3	4
Valor	2	3	6	18	108
	Fijos		Calculables →		

- 13) Dada la siguiente función realizar la prueba de escritorio para el valor 6, y decir claramente qué imprime por pantalla el siguiente main:

```
int main(){
    printf("Prueba 6: %d\n", funcion(6));
    return 0;
}
int funcion(int f){
    if (f == 0){
        return 3;
    }
    if (f == 1){
        return 2;
    }
    if (f == 2){
        return 4;
    }
    return funcion(f-3) + funcion(f-2) + funcion(f-1);
};
```

Explicar luego qué es la recursividad y realizar dos procedimientos recursivos que permitan mostrar los números del 1 al n y del n al 1.

- 14) Explique cada línea del siguiente algoritmo, y muestre que es lo que imprime el algoritmo, justificando claramente su respuesta:

```
#include <stdio.h>
#include <stdlib.h>

void funcion(int *n);

int main(){
    int num = 8;
    funcion(&num);
    return 0;
}
```

```
void funcion(int *n){
    if(*n > 2){
        printf(" %d ", *n);
        *n = *n-1;
        funcion(&n);
    }
}
```

15) Explicar 4 algoritmos de ordenamiento, programar solo 2. Resaltar pro y contras de los 4 algoritmos seleccionados.

16) Crear una función recursiva (o procedimiento recursivo) que genere una búsqueda binaria sobre un array de números enteros.

17) Crear un TDA completo que permita trabajar con números complejos (parte imaginaria + parte real, ambos float). En el TDA tiene que estar definida la función que calcula el módulo del número complejo. Se adjunta el cálculo del módulo, donde a y b son la parte real e imaginario:

$$m = |z| = \sqrt{(a^2 + b^2)}$$

18) Crear un programa completo, bajo el paradigma de TDA que nos permita trabajar con una **PC** y un **Usuario** de la PC. De la PC queremos saber su RAM, marca y memoria. Del usuario, su nombre y su contraseña. Además, queremos guardar en la estructura de la PC el precio de la misma. El precio del PC es la memoria multiplicada por la memoria RAM.

19) Realizar un TDA completo que permita trabajar con un **Electrodomestico**, el mismo tiene un nombre, marca, precio y un número de serie. El número de serie debe ser un array de 8 enteros, aleatorios y sin repetir del 1 al 30. Una vez creado el TDA, desde el **main** crear un electrodoméstico, mostrarlo y mostrar solo el número de serie.

20) Realizar, utilizando el paradigma de TDA, un programa que permita agregar, eliminar y modificar **Empleados** dentro de una **Empresa**. De los empleados nos interesa conocer su nombre, apellido, edad y sueldo. De la empresa queremos saber su dirección, teléfono y rubro. El sueldo de los empleados es 50.000 más su edad multiplicada por 30.

En el **main** deben agregar 4 empleados a una empresa, mostrar la empresa (y sus empleados), eliminar un empleado, y mostrar los empleados ordenados por medio del método de selección (por sueldo).

21) Realizar, utilizando el paradigma de TDA, un programa que permita agregar, eliminar y modificar **Vehículos** dentro de una **Agencia**. De la agencia nos interesa conocer su dirección y su código postal. De los vehículos nos interesa conocer su modelo, color y año, además de cuanto valor ha perdido con los años (en porcentaje). Por cada año que pasó, el auto pierde un valor de 1,2%. En el **main** deben agregar 4 vehículos a una agencia, mostrar los vehículos, eliminar un vehículo, y mostrar los vehículos ordenados por medio del método de inserción (por modelo).

22) Construir un programa que permita agregar, eliminar y modificar **Detalles** de una **Factura**. De la factura nos interesa saber la fecha, la empresa y el total. Mientras que el detalle queremos saber la descripción, cantidad y valor por unidad.

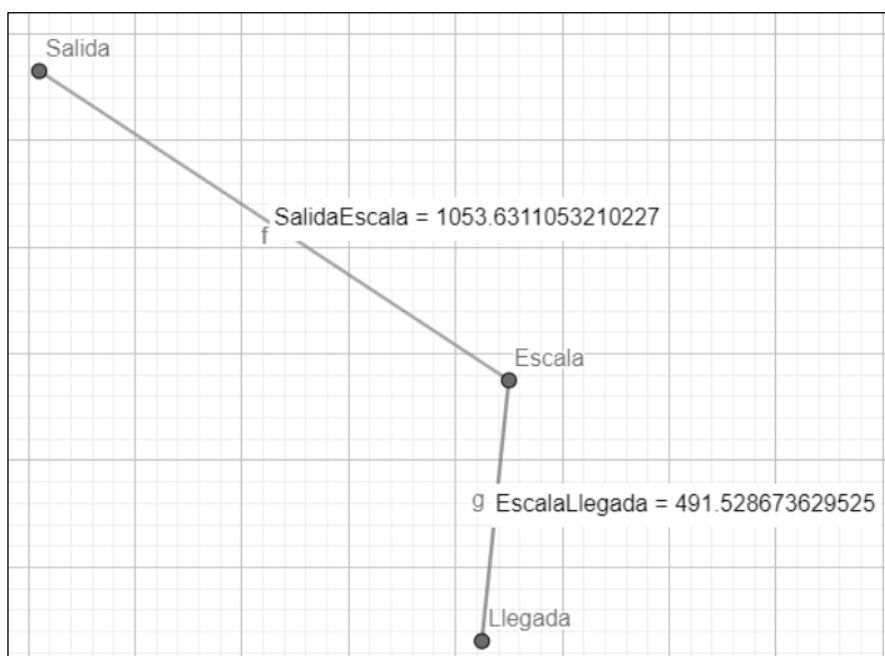
- Anidar correctamente los TDA.
- Agregar detalles a la factura (2 o 3).
- Eliminar un detalle de la factura.
- Modificar un detalle.
- Mostrar el monto final de la factura.

Todo debe ser realizado de la forma más correcta posible en el **main**, solo llamado a funciones y procedimientos, los TDA deben estar completos (solo pongan las pre y pos relevantes no hace falta que pongan todas).

23) Se quiere crear un programa para administrar aerolíneas. Necesitamos trabajar con los datos de **Vuelos**, de cada uno nos interesa saber: su línea, hora de partida, coordenadas de salida (latitud y longitud, o pueden pensarlo como x e y haciendo una analogía con los ejes cartesianos), coordenadas de llegada, coordenadas de la escala intermedia, distancia recorrida y duración del vuelo aproximada. Para calcular la duración del vuelo solo deben saber que por cada 500km de viaje se tardan 0.7hs. Mientras que para calcular la distancia del viaje deberán trabajar con la distancia entre dos puntos del plano.

Fórmula de distancia entre dos puntos:

Para $P_1 = (x_1, y_1)$ y $P_2 = (x_2, y_2)$ se tiene que $d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$



Además, les recordamos que para elevar y sacar raíces pueden utilizar la librería **math.h** con las funciones **pow(float base, int exponente)** y la función **sqrt(float numero)**, respectivamente. Es importante que calculen la distancia total calculando primero la distancia desde el punto de partida hasta el punto de la escala y a eso le suman la distancia del punto de escala al punto de llegada.

En el `main` deberán tener EXACTAMENTE lo siguiente, cambiando sus datos personales obviamente:

```
#include <stdio.h>
#include <stdlib.h>
#include "vuelo.h"

int main(){
    printf("-----FINAL PROGRAMACIÓN - APELLIDO NOMBRE-----\n");
    float vectorSalida[2] = {220.31, 1130.22};
    float vectorEscala[2] = {1100, 550.32};
    float vectorLlegada[2] = {1049.33, 61.41};

    Vuelo v1 = crearVuelo(vectorSalida, vectorLlegada, vectorEscala, "Aerolíneas UNLa",12);
    mostrarVuelo(v1);

    return 0;
}
```

El resultado debe ser:

```
-----FINAL PROGRAMACION 2021 - JULIO - APELLIDO, NOMBRE-----

-----VUELO-----
HORA SALIDA: 12
LINEA: Aerolíneas UNLa
SALIDA: 220.31, 1130.22
ESCALA: 1100.00, 550.32
LLEGADA: 1049.33, 61.41
DURACION APROXIMADA: 2.163224 Horas
DISTANCIA RECORRIDA: 1545.16 KM
```

24) Se tiene el siguiente `archivo.txt`:

```
nico-33-111111
jorge-55-22222
walter-63-333333
nestor-23-444444
```

Crear las estructuras, funciones, procedimientos y/o librerías que crea necesarias para poder levantar el nombre, edad y dni de las personas almacenadas en el archivo.

- 25) Se tienen dos archivos: `primero.txt` y `segundo.txt`. Ambos archivos contienen números enteros, uno sobre el otro, es decir, un número por renglón de forma ordenada. Se pide crear un nuevo archivo llamado `solucion.txt`, en el cual se guarden todos los números de ambos archivos, también ordenados de menor a mayor, pero no pueden estar los números impares. No es válido usar arreglos en este ejercicio. A modo de ejemplo dejo la siguiente secuencia:

<code>primero.txt</code>	<code>segundo.txt</code>	<code>solución.txt</code>
-2	-3	-2
1	-2	-2
2	1	2
2	1	2
3	2	2
7	8	8
12		12

Generar el `.h` de las funciones o procedimientos que cree necesarias para poder hacer un programa de tatetí de juegos por turnos. Explicar con pre y post condiciones cada una de estas funciones y procedimientos.

- 26) ¿Cuáles son las diferencias fundamentales entre una lista, pila y cola? ¿Cuáles son las funciones/procedimientos que caracterizan a estas? ¿Qué virtudes tienen frente a un array?

- 27) ¿Cuáles son las funciones/procedimientos que existen en una Cola, como son las estructuras características de la misma? Utilizar dichas funciones y procedimientos, explicando que se está haciendo en cada instrucción.

- 28) Describir, explicar y codificar cómo serían el/las estructura/s necesarias para trabajar con una cola. Explicar la diferencia entre una cola y una pila.

- 29) ¿Cómo serían la/s estructura/s para crear una lista de char? ¿Qué cambios habría que hacerles a dichas estructuras para que en vez de una lista sea una cola? (codificar las struct).

- 30) Utilizando todas las funciones y/o procedimientos de una lista simplemente enlazada, crear la secuencia numérica 3, 4, 5, -1, 2. Se deberá ejecutar en el `main`.

- 31) De un ejemplo del uso de `*void` en programación. Explique para qué sirve y qué cuidados hay que tener con su uso. Luego genere la estructura que usaría para crear una lista del tipo `*void` (sólo la struct, no el código). Luego explicar por medio de un gráfico y un ejemplo, cómo funciona el algoritmo de inserción para números enteros (no quiero el algoritmo quiero el paso a paso con un ejemplo explicado).